

Evo-RL: Iterative Robot Reinforcement Learning from Selective Experience

Kai Ye, Yifan Jin[†], Qianhang Luo[†], Hongru Zhang[†], Jiedong Zhang,
Shuyuan Hu, Song-Yuan Wu, Jihai Xu, Jie Zhao, Bo Zhao*

Abstract—Improving robot policies from execution feedback requires addressing two challenges: obtaining corrective behavior for difficulties encountered during deployment and identifying useful behavior within accumulated experience. To address these challenges, we propose Evo-RL, an iterative reinforcement learning method that combines human interventions with advantage-guided experience selection. Human interventions provide corrections for difficult states, while advantage estimates prioritize behavior in demonstrations and autonomous rollouts associated with lower execution-time and intervention costs. These complementary sources guide policy fine-tuning, followed by redeployment for further improvement. To support research on learning from these different experience sources, we introduce RW-RL, comprising over 1,000 hours of real-world interaction across four robot embodiments, with action-source metadata and semantic annotations. On real-world precision insertion, Evo-RL improves attempt success rate over RECAP by 30.46%. Our code, model checkpoints, and dataset are publicly available.

I. INTRODUCTION

Early efforts toward generalist robot learning demonstrated that a single language-conditioned policy could learn a broad repertoire of manipulation skills from diverse robot demonstrations [1]. Vision-language-action (VLA) models took this direction further by adapting pretrained vision-language models to generate robot actions, allowing knowledge learned from web-scale data to inform robotic control [2]. Subsequent work has broadened the range of compatible tasks and robot embodiments [3], [4], [5], and investigated more efficient architectures for real-world deployment [6].

However, the capacity to perform diverse tasks alone does not guarantee reliable real-world execution. During deployment, policies trained on successful demonstrations may encounter states that are rare or absent in their training data. With limited supervision on how to act in such states, they may struggle to recover and complete the task [7]. This gap highlights the need for training data that better covers such difficult deployment situations.

Collecting more demonstrations can help improve this coverage, but may also require human operators to repeat behaviors the policy has already learned. Human interventions offer a more targeted alternative: when the policy struggles in out-of-distribution (OOD) states, an operator takes control and guides the robot toward task completion. These human-controlled actions are recorded and used to fine-tune the

policy, concentrating supervision on the difficulties exposed during deployment.

Corrective data address these deployment difficulties, but policy improvement also requires deciding which behavior to learn from the broader pool of experience. As the policy is updated and redeployed, demonstrations, human interventions, and autonomous rollouts accumulate with varying sources and quality. Human corrections provide targeted supervision, while demonstrations and autonomous rollouts contain behavior that may support further refinement. Reinforcement learning offers a criterion for identifying such behavior through advantage estimation, which measures an action’s expected return relative to the policy’s baseline at the same state.

To address both the need for corrective experience and the selection of useful behavior, we propose Evo-RL, an iterative real-world reinforcement learning method inspired by RECAP [8]. Starting from a demonstration-trained policy, Evo-RL uses human interventions to refine behavior in difficult states and advantage estimates to select and prioritize experience from demonstrations and autonomous rollouts. The updated policy is redeployed to collect feedback for the next round, linking experience collection and selection across successive policy updates.

Studying how these experience sources contribute to policy improvement requires data that distinguish human corrections from autonomous actions and retain their task context and outcomes. To support this research, we introduce RW-RL, which organizes demonstrations, human-intervention data, and autonomous rollouts with action-source metadata and semantic annotations. This organization enables analysis of how different kinds of experience can support iterative policy learning.

Our main contributions are as follows:

- We propose Evo-RL, a real-world reinforcement learning method that alternates between human-intervention refinement and advantage-guided experience selection to improve policy reliability across successive collection-and-training rounds. On real-world precision insertion, Evo-RL improves attempt success rate (ASR) over RECAP by 30.46 percentage points.
- We present RW-RL, a real-world robot dataset designed for iterative policy learning. It spans over 1,000 hours of interactions across four robot embodiments. Equipped with action-source metadata and semantic annotations, RW-RL facilitates research into how human guidance and autonomous experience jointly drive policy improvement.

[†]Equal Contribution. *Corresponding author.

Code: <https://github.com/MINT-SJTU/Evo-RL>

Dataset: <https://huggingface.co/datasets/MINT-SJTU/RW-RL-Dataset>

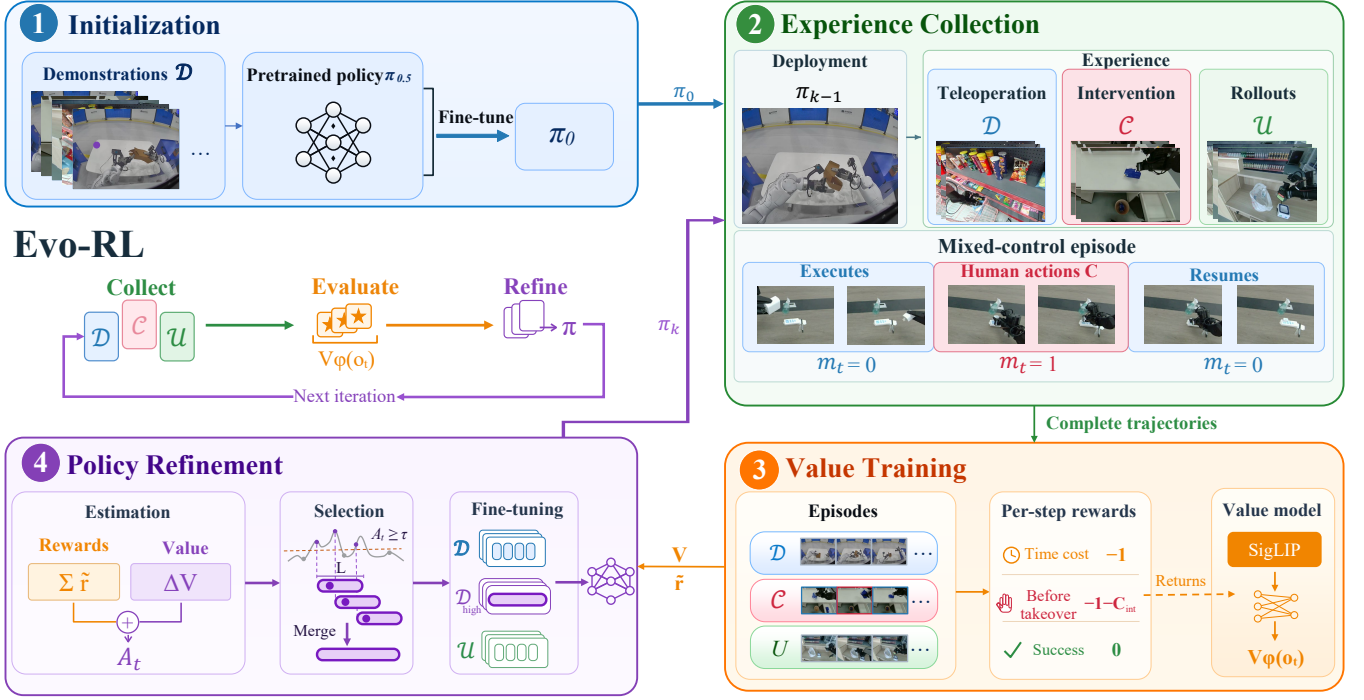


Fig. 1. **Overview of the Evo-RL method.** The current policy is deployed to collect demonstrations, human interventions, and autonomous rollouts. Complete trajectories are used to train a value model whose estimates guide policy refinement. The updated policy is then redeployed in the next iteration.

II. RELATED WORK

A. Vision-Language-Action Model and Robot Datasets

Large-scale robot datasets collectively cover diverse household, office, and industrial manipulation tasks across single-arm, dual-arm, and humanoid platforms [9], [10], [11], [12], [13]. This diversity provides a foundation for VLA pretraining, enabling policies to learn reusable manipulation skills across tasks and robots [13]. Building on these skills, models such as OpenVLA can adapt to new tasks and robot setups through fine-tuning on additional demonstrations [3]. However, even after fine-tuning, policies may still fail in situations poorly covered by datasets. Human interventions provide targeted corrections in these situations, yet the resulting corrective experience is less systematically captured and organized.

B. human interventions and Data Aggregation

Out-of-distribution (OOD) situations refer to scenarios that differ from those encountered during training and may therefore challenge the policy’s ability to generalize. During deployment, human interventions can be introduced in such OOD situations, not necessarily only after an execution error occurs. These interventions provide corrective demonstrations that can be incorporated into the training data, helping the policy improve its responses when similar situations arise in the future. Dataset Aggregation (DAgger) formalizes this process by querying expert actions for states visited by the current policy and aggregating the resulting state–action pairs into the training set [7].

Building on this paradigm, subsequent research focuses on two related questions: when human interventions should be triggered and how the resulting correction data should be incorporated into policy learning. Some methods let human operators decide when to intervene [14], whereas others use state novelty or failure risk to determine when to request human assistance [15]. Because intervention samples often constitute only a small fraction of the collected data, their influence on policy updates can be limited. IWR addresses this imbalance by increasing the relative weight of intervention samples, while using non-intervention samples to regularize policy updates and help retain previously learned skills [16]. However, corrective action labels alone do not explicitly indicate which states are likely to lead to failure or which trajectories are of higher quality. Incorporating such evaluations can provide additional supervisory signals for further policy improvement.

C. Real-World RL and Iterative Policy Improvement

Reinforcement learning enables robots to improve their policies through interaction with the environment. Early work on RL for VLA models focused on offline preference alignment[17], [18], such as GRAPE [18], which employs Direct Preference Optimization (DPO) [19] to align robot trajectories with human intent. More recent approaches incorporate experience collected during deployment into VLA post-training. RECAP trains a value function on demonstrations, human interventions and autonomous rollouts then uses advantage-based indicators to condition policy training and guide iterative improvement[8]. To reduce the cost of online adaptation, RLT compresses latent representations into



Fig. 2. Overview of RW-RL, a large-scale real-world robot learning dataset spanning diverse embodiments and tasks, with hierarchical semantic annotations, synchronized multimodal observations and control signals, and over 1,000 hours of real-world interaction data.

an RL token and trains a lightweight actor-critic to refine action chunks while keeping the VLA frozen[20]. Beyond action-level optimization, LaST-R1 extends reinforcement learning to the latent space, jointly optimizing reasoning and execution[21]. Our Evo-RL is motivated by RECAP, as both methods combine human corrections with autonomous rollouts to realize value-guided iterative policy improvement.

III. METHOD

We organize demonstration-based learning, intervention-based refinement, and autonomous optimization into a data loop. Starting from demonstration data, we initialize the policy by fine-tuning a pretrained VLA model. Next, human interventions collected during deployment are used for policy fine-tuning to improve performance in challenging scenarios. Once the policy is able to complete tasks reliably, we leverage deployment experience to perform value-guided policy updates for further gains in efficiency and quality. Algorithm 1 summarizes this loop and the policy updates within each iteration.

A. Demonstration-Based Policy Initialization

At time step t , the policy receives an observation $o_t = (I_t, s_t)$ and a task instruction ℓ , where I_t denotes the camera images and s_t the robot’s proprioceptive state. The policy models a conditional distribution over action chunks:

$$\mathbf{a}_t = (a_t, \dots, a_{t+H-1}) \sim \pi_\theta(\cdot \mid o_t, \ell), \quad (1)$$

where H is the chunk length and θ denotes the policy parameters. Following the architecture of $\pi_{0.5}$ [22], our policy combines a PaliGemma vision-language backbone with a flow-matching action expert to generate action chunks conditioned on visual observations, proprioceptive state, and task instructions. We train the policy on teleoperated demonstrations D_{demo} using the flow-matching objective.

Algorithm 1 Evo-RL: Iterative Robot Reinforcement Learning with Selective Experience

Input: Policy π_K , demonstrations D_{demo} , human corrections D_{interv} , rounds R .

Output: Final policy π_{K+1+2R} .

1: **Demonstration-Based Initialization**

2: $\pi_{K+1} \leftarrow \text{SFT}(\pi_K, D_{demo})$.

3: Initialize D_{auto} , D'_{auto} , D'_{demo} , and D_{train} as empty sets.

4: **for** $k = K + 1, K + 3, \dots, K + 2R - 1$ **do**

5: **1. Policy Refinement with Human Interventions**

6: Deploy π_k to collect human corrections and update D_{interv} .

7: $\pi_{k+1} \leftarrow \text{SFT}(\pi_k, \text{Mix}(D_{demo}, D_{interv}))$.

8: **2. Advantage-Guided Policy Fine-Tuning**

9: Optionally deploy π_{k+1} to collect autonomous rollouts and update D_{auto} .

10: Train V_ϕ on D_{demo} , D_{interv} and D_{auto}

11: Compute $\hat{A}_t$ on D_{demo} and D_{auto} .

12: Select high-advantage segments as D'_{demo} and D'_{auto} .

13: $D_{train} \leftarrow \text{Mix}(D_{demo}, D'_{auto}, D_{interv})$.

14: $\pi_{k+2} \leftarrow \text{SFT}(\pi_{k+1}, D_{train})$.

15: **end for**

▷ Segments in $D'_{demo} \subseteq D_{demo}$ are sampled with elevated probability.

For each demonstration sample $(o_t, \ell, \mathbf{a}_t) \sim D_{demo}$, we sample standard Gaussian noise ϵ with the same shape as $\mathbf{a}_t$, and a flow time $\tau \in [0, 1]$. The noisy action chunk is constructed as

$$x_t^\tau = (1 - \tau)\mathbf{a}_t + \tau\epsilon. \quad (2)$$

The action expert v_θ predicts the corresponding velocity $\epsilon - \mathbf{a}_t$, conditioned on o_t and ℓ . The objective function can be written as

$$\mathcal{L}_{demo}(\theta) = \mathbb{E}[\text{MSE}(v_\theta(x_t^\tau, o_t, \ell, \tau), \epsilon - \mathbf{a}_t)], \quad (3)$$

While this objective enables effective behavioral cloning, the model’s capability and generalization remain inherently

bounded by the quality, scale, and diversity of the demonstration data. This fine-tuning produces the initial task policy π_0 , which is subsequently deployed to collect experience for further improvement.

B. Policy Refinement with Human Interventions

The initial policy may still struggle in challenging scenarios. We therefore use human-supervised rollouts to collect corrective actions. An operator monitors task execution and intervenes when the policy encounters states poorly covered by the demonstrations and struggles to recover or make further progress. After each correction, the robot resumes autonomous operation. For an intervention trajectory indexed by i , we represent the information collected at each frame t as

$$S_{i,t} = (\ell_i, o_{i,t}, a_{i,t}, m_{i,t}),$$

where ℓ_i is the task instruction, $o_{i,t}$ contains images and proprioceptive state, and $a_{i,t}$ is the executed action. We introduce the control-source label $m_{i,t}$ to distinguish human corrections from policy actions, with 1 indicating human control and 0 indicating policy control. A trajectory of length T_i is written as

$$\xi_i = (S_{i,1}, \dots, S_{i,T_i}),$$

with $y_i \in \{0, 1\}$ indicating whether the task succeeds. We retain human-controlled data from successful intervention trajectories to construct the dataset D_{interv} :

$$D_{\text{interv}} = \{S_{i,t} \mid y_i = 1, m_{i,t} = 1\}.$$

We preserve the temporal order and segment boundaries of these data and sample training action chunks only within contiguous human-controlled segments. Policy-controlled portions are excluded from D_{interv} to avoid learning from actions that are likely to cause failures or prolong task completion. During training, we fine-tune the current policy with the same loss function, sampling from D_{interv} and D_{demo} with probabilities $p \in (0, 1)$ and $1 - p$, respectively. The updated policy is then redeployed to identify remaining difficulties and collect further corrections. This cycle continually incorporates human corrections into training, allowing the policy to learn from its mistakes and improve task performance over successive updates.

C. Reinforcement Learning for Policy Refinement

Demonstration training and intervention-based refinement provide an initial policy and corrective supervision. Further improvement requires finer-grained signals beyond episode-level success or failure to identify promising action segments within the accumulated experience. We use rewards that account for execution time and human takeovers to learn a value model and estimate segment advantages. These estimates guide experience selection and sampling for policy fine-tuning in each round.

1) Reward Design with Time and Intervention Costs:

To account for both execution time and reliance on human corrections, we construct rewards from per-frame costs and takeover events. Using the label m_t introduced in the previous subsection, we identify a takeover as

$$b_{t+1} = \mathbf{1}[m_t = 0, m_{t+1} = 1]. \quad (4)$$

For a successful trajectory, let T denote the index of the terminal observation and set $b_T = 0$. The frame-level reward is

$$r_t = \begin{cases} 0, & t = T, \\ -1 - C_{\text{int}}b_{t+1}, & t < T, \end{cases} \quad (5)$$

where $C_{\text{int}} \geq 0$ is the additional cost of a human takeover.

A takeover typically indicates that the policy's behavior requires correction. We therefore treat it as negative feedback and assign the penalty to the policy transition immediately preceding the takeover, rather than to the subsequent human corrections. The unit cost at each nonterminal frame encourages faster task completion. Together, these terms favor trajectories with lower execution time and less dependence on human assistance.

2) *Value Function Learning from Trajectory Returns:* For each task, we train a vision-only value model with parameters ϕ independently of the policy. Given camera images I_t , it predicts a categorical distribution $p_\phi(c \mid I_t)$ over 201 bins whose centers v_c are evenly spaced in $[-1, 0]$.

The return of frame t is defined as $G_t = \sum_{u=t}^T r_u$. We normalize it by a fixed, task-specific scale $Z_\ell > 0$, giving a training target $G_t/Z_\ell \in [-1, 0]$. Returns closer to zero indicate a lower combined cost of remaining execution time and human takeovers.

Each normalized return is encoded as a two-hot target $q_t(c)$, with weights determined by linear interpolation between adjacent bin centers. We train the model on sampled trajectory frames using the cross-entropy loss

$$\mathcal{L}_V(\phi) = -\mathbb{E}_{\xi,t} \left[\sum_{c=1}^{201} q_t(c) \log p_\phi(c \mid I_t) \right]. \quad (6)$$

The scalar value estimate is the probability-weighted mean of the bin centers:

$$V_\phi(I_t) = \sum_{c=1}^{201} p_\phi(c \mid I_t) v_c. \quad (7)$$

3) *Advantage-Guided Policy Fine-Tuning:* Using the trained value model, we estimate the advantage of an action sequence as

$$\hat{A}_t = \sum_{j=0}^{n_t-1} \tilde{r}_{t+j} + V_\phi(o_{t+n_t}, \ell) - V_\phi(o_t, \ell). \quad (8)$$

Here, $\tilde{r}_t = r_t/Z_\ell$ is the normalized reward, and $n_t = \min(n, T-t)$ limits the lookahead window of at most $n \leq H$ frames to the remaining trajectory. The terminal value is fixed to $V_\phi(o_T, \ell) = 0$.

In each training round, we use estimated advantages to select promising segments from D_{demo} and available D_{auto} .

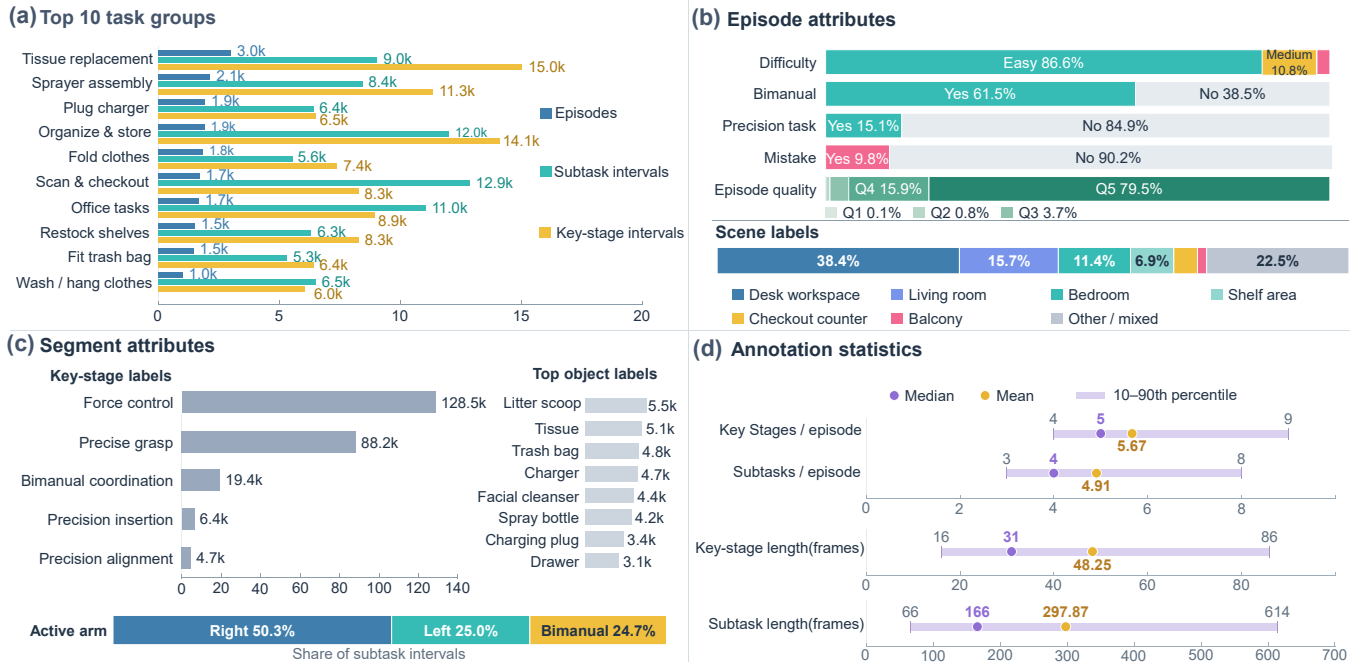


Fig. 3. Annotation statistics of RW-RL, summarizing task coverage, episode-level attributes, segment-level labels, and the counts and durations of annotated subtasks and key stages.

These segments are mixed with human corrections to fine-tune the policy with the same flow-matching loss. Selection and sampling details are given in Section V-E.

IV. RW-RL DATASET

Iterative policy optimization with Evo-RL continuously accumulates human corrections and autonomous deployment experiences throughout closed-loop learning. The real-world interaction trajectories collected across all optimization stages form RW-RL, a comprehensive robot dataset tailored for iterative policy improvement. RW-RL integrates three complementary data sources: teleoperated demonstrations, human-intervention segments, and autonomous policy rollouts. Figure 2 summarizes four key dimensions of RW-RL: robot embodiment diversity, long-horizon real-world executions, frame-aligned semantic annotations, and multi-source collection regimes for iterative policy learning.

A. Embodiments, Modalities, and Collection Regimes

The dataset contains more than 1,000 hours of real-world interaction collected on four robot embodiments: AgiBot G2, Galaxea R1Lite, AgileX PiperX, and SO-101. These platforms span industrial mobile manipulation, modular dual-arm systems, and low-cost tabletop learning.

G2 provides eight RGB video streams (top, top-back, paired fisheye and stereo views, and two wrist views), together with joint states, end-effector poses and wrenches, gripper/head/waist/base states, and actions. R1Lite provides four RGB streams (top, top-right, and two wrist views), together with arm joint positions and velocities, gripper/torso/chassis states, chassis IMU readings, end-effector poses, and actions. The PiperX and SO-101 platforms are

dual-arm systems. Both are fitted with an external RGB camera and a pair of wrist cameras. PiperX uses 14-dimensional state and action vectors, while SO-101 employs 12-dimensional counterparts.

Beyond embodiment diversity, RW-RL also covers a broad range of real-world task settings. The data span nine scene domains: store shelves, industrial production, commercial kitchens, home spaces, care environments, laboratories, warehouses, hotels, and education or training settings. Across these domains, the dataset instantiates more than 30 task templates, including object sorting, shelf replenishment, scanning and bagging, precision insertion, garment folding, and box stacking.

RW-RL further distinguishes episodes by how the actions are generated during collection. Demonstrations record human execution; autonomous episodes record policy execution; and intervention episodes combine both, with frame-level controller labels identifying human correction segments.

B. Data Organization and Hierarchical Semantic Annotations

Each episode in the dataset links visual observations, robot states, and executed actions through episode identifiers and timestamps. These records are accompanied by semantic annotations at three levels of granularity: dataset, episode, and frame (Fig. 3). Dataset-level fields specify the task, embodiment, difficulty, objects, and coordination or precision requirements. Episode-level fields describe instructions, scenes, outcomes, execution quality, and mistakes. They also include action-source metadata, which categorizes episodes into three types: teleoperated demonstrations, autonomous

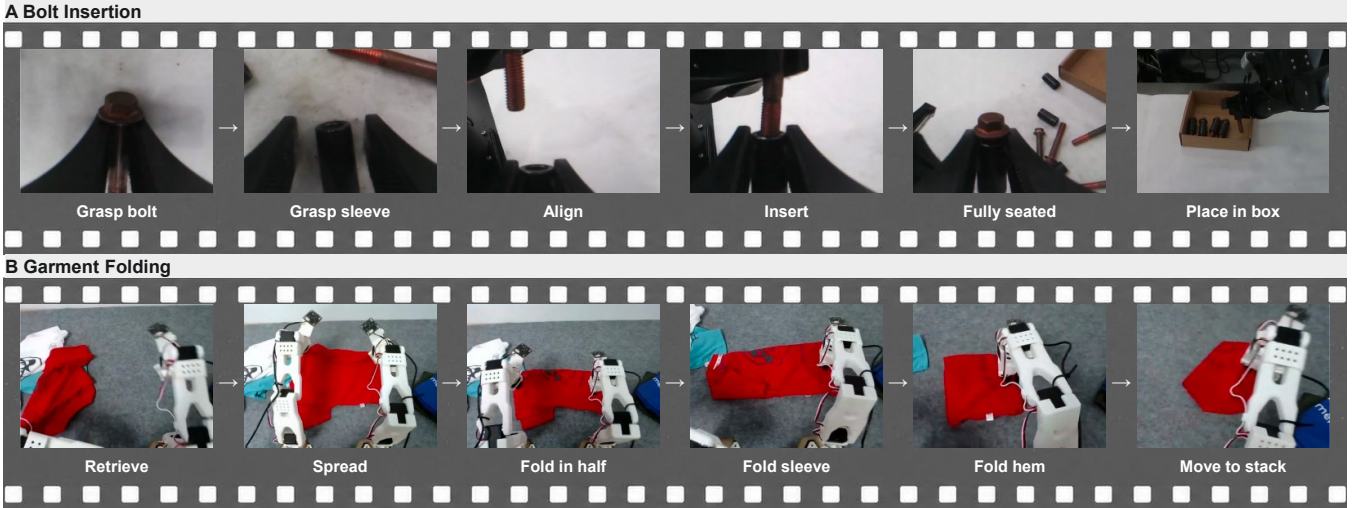


Fig. 4. Task sequences in RW-RL: (A) bolt insertion and (B) garment folding. Each row shows one episode in chronological order from left to right.

policy rollouts, and human-intervention episodes. Frame-level annotations consist of subtask intervals, along with frame-wise controller labels that identify human and policy actions. A subtask interval records its start and end frames, local instruction, manipulated objects, and active arm, and marks operation attributes such as bimanual coordination. All these annotations are synchronized under a unified timestamp system for straightforward cross-reference.

V. EXPERIMENTS

We first assess whether intervention data are more effective than additional demonstrations under the same data budget (Table I). Beyond this data-efficiency comparison, task-specific metrics are used to evaluate policy improvements across successive training stages (Tables II). Representative sequences from the dataset illustrate the progression of each task in Figure 4. We also describe our value-model choice and advantage-guided training-data mixture.

A. Experimental Setup and Task Definitions

We conduct experiments on two robot platforms: PiperX and SO-101. Policies are initialized from $\pi_{0.5}$ and deployed with real-time chunking (RTC) [23].

Each trial has a five-minute time limit. A trial is considered a failure if the task is not completed within this limit. The setups for the two manipulation tasks are described below.

PiperX insertion. Five screw-sleeve pairs are placed on the tabletop. The robot selects, grasps, aligns, and inserts each screw. Either arm can insert the screw into the sleeve held by the other. Each unit U_j corresponds to inserting one screw into a sleeve.

SO-101 folding. Five T-shirts are loosely piled on the desk. The robot retrieves and folds them sequentially. Units U_1 – U_5 denote retrieval, spreading, folding in half, folding the sleeves, and folding the hem toward the collar. If the garment enters an erroneous state, an operator manually restores it to the state before the failed stage without fully smoothing it out.

B. Evaluation Metrics

For PiperX insertion and SO-101 folding, we evaluate the trained models using two metrics. ASR is the total number of successful attempts divided by the total number of attempts, aggregated across U_1 – U_5 and all trials, reported in %. Throughput (TP) is the number of successful tasks per hour. Each U_j cell represents one individual stage: either a PiperX insertion unit or one SO-101 folding stage. It reports the number of successful completions for that stage and the number of attempts made for that stage. An attempt count of 0 indicates that the trial timed out before reaching this stage. N is the success target per unit or stage, at which evaluation stops: $N = 10$ for PiperX and $N = 20$ for SO-101.

C. Data Efficiency of Human Interventions

To evaluate whether targeted human corrections yield greater performance gains than equivalently scaled teleoperated demonstrations under a fixed data-collection budget, we compare two data collection regimes with the same time budget: one collects demonstrations only, while the other reserves part of the budget (an hour) for human corrections. If interventions are more efficient, the same budget should translate into higher policy performance than additional demonstrations alone. We benchmark a demonstrations-only baseline (SFT) against SFT + Int., which substitutes one hour of teleoperated demonstrations with human-intervention data. The two regimes are evaluated as paired comparisons under identical total budgets, scaled from 2 to 16 hours and trained from the same pretrained checkpoint. With a four-hour budget, adding one hour of human-intervention data raises ASR from 27.03% to 51.52%, and TP from 12.00 to 20.40, compared with the baseline (Table I). The eight-hour mixture achieves an ASR of 78.00%, comparable to 16-hour SFT (78.26%), while reaching a similar throughput of 53.04 versus 53.27 completed stages per hour.

These gains can be attributed to a fundamental difference between demonstrations and interventions in how they cover

TABLE I

TARGETED INTERVENTIONS IMPROVE PIPERX ASR UNDER MATCHED ACTIVE HUMAN-CONTROL BUDGETS, WHICH INCLUDE DEMONSTRATION AND INTERVENTION TIME. EACH METHOD USES $N = 10$ PLANNED INSTANCES PER UNIT.

Budget	Method	Successes / attempts					ASR (%)	TP
		U_1	U_2	U_3	U_4	U_5		
2 h	SFT	0/18	0/0	0/0	0/0	0/0	0.00	0.00
	SFT + Int.	5/12	1/1	0/0	0/0	0/0	46.15	7.20
4 h	SFT	5/23	4/7	1/7	0/0	0/0	27.03	12.00
	SFT + Int.	7/13	5/14	3/3	2/2	0/1	51.52	20.40
8 h	SFT	9/21	9/14	8/23	8/19	4/10	43.68	50.17
	SFT + Int.	9/11	9/10	8/12	7/10	6/7	78.00	53.04
16 h	SFT	9/12	8/9	8/8	6/12	5/5	78.26	53.27

TABLE II

COMBINING CORRECTIONS WITH RL IMPROVES PIPERX ASR AND THROUGHPUT BEYOND EITHER COMPONENT ALONE; A SECOND ROUND YIELDS FURTHER GAINS. EACH METHOD USES $N = 10$ PLANNED INSTANCES PER PIPERX UNIT AND $N = 20$ PER SO-101 FOLDING STAGE.

Task	Method	Successes / attempts					ASR (%)	TP
		U_1	U_2	U_3	U_4	U_5		
SO-101 folding	SFT	20/20	17/76	16/18	16/21	16/18	55.56	75.78
	Evo-RL	20/20	19/36	18/22	18/22	17/20	76.67	90.00
PiperX insertion	SFT	7/20	3/15	0/9	0/0	0/0	22.73	12.00
	SFT + Int.	9/11	9/12	8/10	6/8	5/6	78.72	47.98
	SFT + RL	10/25	8/21	5/14	3/10	1/3	36.99	33.69
	RECAP	8/13	6/10	5/11	4/5	3/6	57.78	35.54
	Evo-RL	10/10	10/10	10/12	10/11	5/8	88.24	60.61
	Evo-RL (R2)	10/10	10/10	10/11	9/10	8/10	92.16	67.14

the state space. Successful teleoperation may rarely cover states induced by policy errors, whereas interventions supply corrective actions in these states. Their value lies in relevance to the current policy’s failures. Since the policy executes autonomously for most of the episode, human supervision intervenes only at a few critical moments, supplying precisely the data the model most needs and thereby increasing the value of the collected training data.

D. Intervention and Real-Robot RL

Beyond the data-efficiency comparison, we evaluate how human interventions and reinforcement learning jointly support continued policy improvement. We conduct ablation experiments on PiperX to assess the contribution of each training stage (Table II). All variants start from the same policy trained on demonstrations for 2.5 epochs (SFT). To evaluate human interventions alone, we fine-tune the policy on demonstrations mixed with one hour of human interventions, sampling interventions with probability $p = 0.5$ (SFT + Int.). Alternatively, we apply advantage-guided selection to the baseline policy using only demonstration data (SFT + RL): advantages computed from the demonstrations identify high-advantage segments, which are then sampled with elevated probability during fine-tuning. The combined setting (Evo-RL) integrates intervention-based fine-tuning with advantage-guided sampling: it mixes original demonstrations, high-advantage demonstration segments sampled with elevated probability, and human-intervention data for policy fine-tuning. High-advantage segments from successful

autonomous rollouts are also included in this round. Finally, we repeat the data collection and training cycle to evaluate the benefits of a second iteration (Evo-RL R2).

PiperX insertion. The ablations in Table II examine how human corrections and reinforcement learning contribute to policy improvement. Both SFT + Int. and SFT + RL improve over SFT, with human corrections providing the larger gain. Combining the two in Evo-RL yields the highest ASR and TP among the first-round variants, raising ASR from 78.72% for SFT + Int. to 88.24% and improving TP by 26.3%. These results support complementary roles for corrective experience and advantage-guided refinement: corrections supply behavior for difficulties encountered during deployment, while advantage-guided sampling directs further training toward promising behavior in the available data.

Beyond these ablations, first-round Evo-RL outperforms RECAP on PiperX, achieving 88.24% versus 57.78% ASR and 60.61 versus 35.54 TP. Policy fine-tuning takes approximately 4 h 52 min for Evo-RL (1,511 steps), compared with an estimated 64 h for RECAP (102,111 steps, 2.5 epochs), both using eight A800 80 GB GPUs.

Statistics show that the human-control time fraction decreases from 15.30% for SFT to 9.56% for Evo-RL. Together with the higher ASR and TP, this indicates that improved task performance coincides with a smaller share of recorded execution under direct human control. A second collection-and-training round further raises ASR from 88.24% to 92.16% and TP from 60.61 to 67.14, with U_5 completions increasing from five to eight. Thus, another round can improve execu-

tion beyond the first update.

SO-101 folding. The complete Evo-RL method also improves over SFT on folding, raising ASR from 55.56% to 76.67% and TP from 75.78 to 90.00. At the spreading stage (U_2), successful completions increase from 17 to 19 while attempts decrease from 76 to 36. This substantial reduction in repeated attempts helps explain the higher aggregate ASR and accompanies improved throughput, despite a more modest increase in successful stage completions.

E. Value Learning and Advantage-Guided Sampling

Our value models are task-specific and vision-only. Each model consists of a SigLIP visual encoder followed by an MLP output head, and is trained by randomly sampling frames from demonstrations $\mathcal{D}_{\text{demo}}$, human-intervention episodes $\mathcal{D}_{\text{interv}}$, and autonomous rollouts $\mathcal{D}_{\text{auto}}$. **Value training takes approximately 2 h 19 min on three A100 GPUs for Evo-RL, compared with 3 h 46 min on eight A800 80 GB GPUs for RECAP.**

The same value model and advantage computation apply to demonstrations and successful autonomous rollouts. We jointly select nonterminal starts with positive advantages in the top 10% across both sources, merging overlapping or adjacent retained chunks. Selected autonomous segments join the original demonstrations and human corrections for fine-tuning; selected demonstration segments receive increased sampling weight.

Figure 5 plots the value and advantage estimates over a screw-insertion episode. Gold dots mark the selected starts, and the overlaid images show the corresponding observations. Which segments are retained for training is determined by the advantage at each start, not by high state value alone. The selected states consistently correspond to critical transition phases of the task: grasping the target object, manipulating it (inserting the screw into the sleeve), and placing it at the target location. These are precisely the points where small errors most affect downstream execution, and where the policy most needs fine-grained supervision. Notably, states near task completion often exhibit high value yet low advantage, confirming that value alone is insufficient for selecting informative segments. This observation indicates that advantage estimation successfully isolates the key decision points of a task, allowing us to construct focused and efficient fine-tuning datasets in each iteration and thereby reduce training cost and time.

VI. CONCLUSION

We presented Evo-RL, an iterative robot reinforcement learning method that combines human corrections with advantage-guided experience selection. Human corrections target difficult states encountered during deployment, while value and advantage estimates derived from execution-time and human-takeover costs guide experience selection and sampling for policy refinement. RW-RL records demonstrations, interventions, and autonomous executions with provenance and outcome annotations to support this iterative

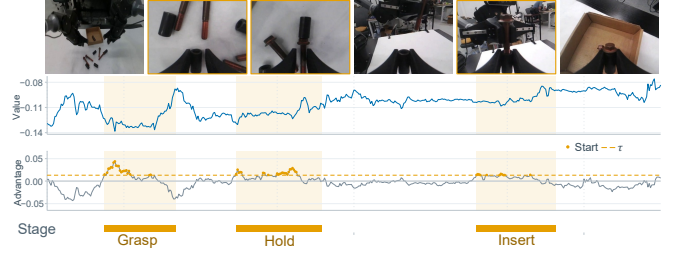


Fig. 5. **Visual value estimation and segment selection.** The visual value function predicts normalized remaining returns to evaluate task states and guide advantage-based training-data selection. Blue and gray curves show value and advantage estimates; gold dots denote selected starts, and the dashed line marks the cutoff τ_A . Gold regions indicate retained segments formed by merging overlapping or adjacent 20-frame forward windows, with bordered images showing corresponding observations.

learning process. Under our evaluation protocol, incorporating corrections improves the use of human supervision, and combining corrections with reinforcement learning improves attempt success rate and throughput on insertion and folding. PiperX ablations show gains over either component alone, with further improvements in a second round. These results support combining targeted corrections with experience selection across successive learning rounds to improve robot policy reliability and execution efficiency.

REFERENCES

- [1] A. Brohan, N. Brown, J. Carbajal *et al.*, “RT-1: Robotics transformer for real-world control at scale,” in *RSS*, 2023.
- [2] B. Zitkovich, T. Yu, S. Xu *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *CoRL*, 2023.
- [3] M. J. Kim, K. Pertsch, S. Karamcheti *et al.*, “OpenVLA: An open-source vision-language-action model,” in *CoRL*, 2024.
- [4] K. Black, N. Brown, D. Driess *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” in *RSS*, 2025.
- [5] W. Wu, F. Lu, Y. Wang *et al.*, “A pragmatic VLA foundation model,” *arXiv preprint arXiv:2601.18692*, 2026.
- [6] T. Lin, Y. Zhong, Y. Du *et al.*, “Evo-1: Lightweight vision-language-action model with preserved semantic alignment,” in *CVPR*, 2026.
- [7] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *AISTATS*, 2011.
- [8] Physical Intelligence, A. Amin, R. Aniceto *et al.*, “ $\pi_{0.6}^*$: a VLA that learns from experience,” *arXiv preprint arXiv:2511.14759*, 2025.
- [9] H. R. Walke, K. Black, T. Z. Zhao *et al.*, “BridgeData V2: A dataset for robot learning at scale,” in *CoRL*, 2023.
- [10] A. Khazatsky *et al.*, “DROID: A large-scale in-the-wild robot manipulation dataset,” in *RSS*, 2024.
- [11] H.-S. Fang, H. Fang, Z. Tang *et al.*, “RH20T: A comprehensive robotic dataset for learning diverse skills in one-shot,” in *ICRA*, 2024.
- [12] K. Wu, C. Hou, J. Liu *et al.*, “RoboMIND: Benchmark on multi-embodiment intelligence normative data for robot manipulation,” in *RSS*, 2025.
- [13] Open X-Embodiment Collaboration *et al.*, “Open X-Embodiment: Robotic learning datasets and RT-X models,” in *ICRA*, 2024.
- [14] M. Kelly, C. Sidrane, K. Driggs-Campbell *et al.*, “HG-DAGger: Interactive imitation learning with human experts,” in *ICRA*, 2019.
- [15] R. Hoque, A. Balakrishna, E. Novoseller *et al.*, “ThriftyDAGger: Budget-aware novelty and risk gating for interactive imitation learning,” in *CoRL*, 2021.
- [16] A. Mandlekar, D. Xu, R. Martín-Martín *et al.*, “Human-in-the-loop imitation learning using remote teleoperation,” *arXiv preprint arXiv:2012.06733*, 2020.
- [17] H. Zhang, Z. Zhuang, H. Zhao *et al.*, “ReinboT: Amplifying robot visual-language manipulation with reinforcement learning,” in *ICML*, 2025.

- [18] Z. Zhang, K. Zheng, Z. Chen *et al.*, “GRAPE: Generalizing robot policy via preference alignment,” in *ICRA FMNS Workshop*, 2025.
- [19] R. Rafailov, A. Sharma, E. Mitchell *et al.*, “Direct preference optimization: Your language model is secretly a reward model,” in *NeurIPS*, 2023.
- [20] C. Xu, J. T. Springenberg, M. Equi *et al.*, “RL Token: Bootstrapping online RL with Vision-Language-Action models,” *arXiv preprint arXiv:2604.23073*, 2026.
- [21] H. Chen, J. Liu, Z. Yan *et al.*, “LaST-R1: Reinforcing robotic manipulation via adaptive physical latent reasoning,” *arXiv preprint arXiv:2604.28192*, 2026.
- [22] K. Black, N. Brown, J. Darpinian *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” in *CoRL*, 2025.
- [23] K. Black, M. Galliker, and S. Levine, “Real-time execution of action chunking flow policies,” in *NeurIPS*, 2025.